

硬件描述语言 Verilog

*实验实际要求

实验学时只有8小时，verilog知识主要靠自学，实验安排4个，考虑到实际情况，做的相对简单一点，所以主要掌握以下一些知识：

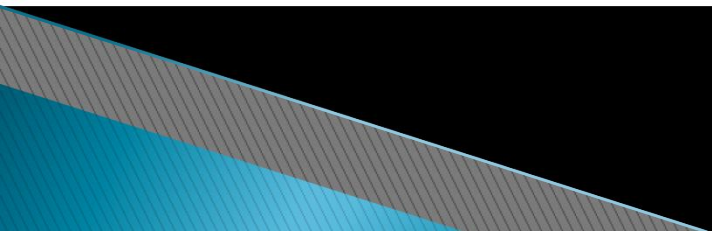
- } 1. 一个verilog模块怎么定义
- } 2. 常用的数据表示，常量（整数integer），一些常用运算符
- } 3. 最常用的数据类型：wire, reg
- } 4. 常用的逻辑功能描述语句：assign, always, if-else,
case, for, =（赋值）
- } 5. 元件例化
- } 6. 仿真文件建立（看操作指南）
- } 7. 硬件实现：引脚定义（看操作指南）

verilogHDL内容概要

- } 一、什么是VerilogHDL?
- } 二、Verilog的词法
- } 三、Verilog的语法
- } 四、Verilog描述组合逻辑电路
- } 五、Verilog描述时序电路

什么是VerilogHDL ?

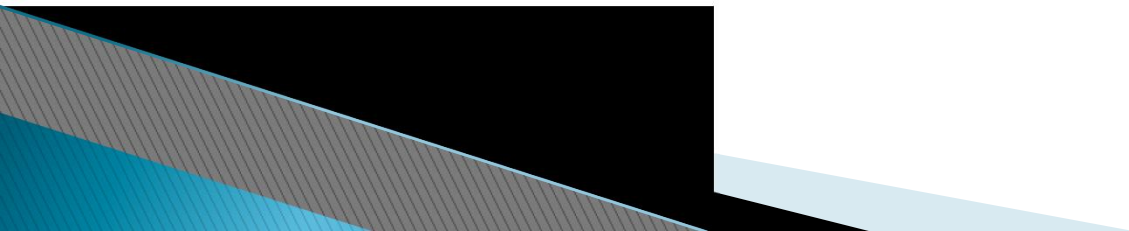
- } **定义：** Verilog HDL是一种用于数字逻辑电路设计的硬件描述语言（Hardware Description Language）。
- } 使用VerilogHDL可以像设计软件一样设计硬件系统



设计数字系统的基本方法

传统的设计方法： 原理图

现代的设计方法： 硬件描述语言
(HDL, Hardware Description Language)



传统原理图

不适合复杂数字系统设计

已有的器件不一定适用

不能接受

如何解决？

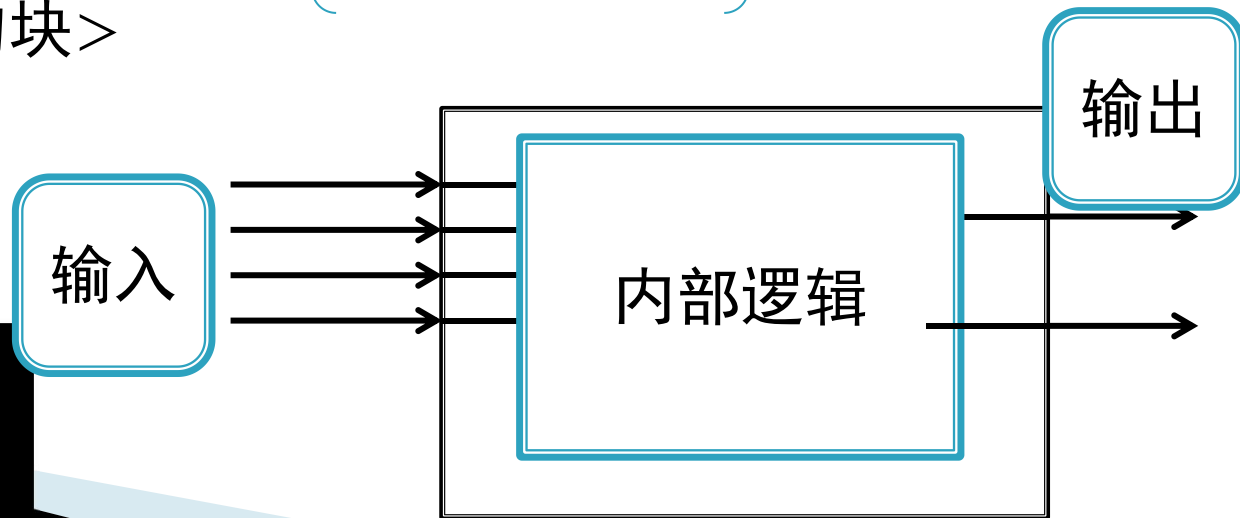
} 可编程逻辑器件的出现

} HDL的发展

Verilog模块基本结构

```
} module <模块名>(口1, 口2, .....);  
}   <I/O端口说明>  
}  
}   .....  
}   <内部信号说明>  (变量声明)  
}  
}   .....  
}   <assign语句>  
}   <实例元件语句>  
}   <结构说明语句块>  
} endmodule
```

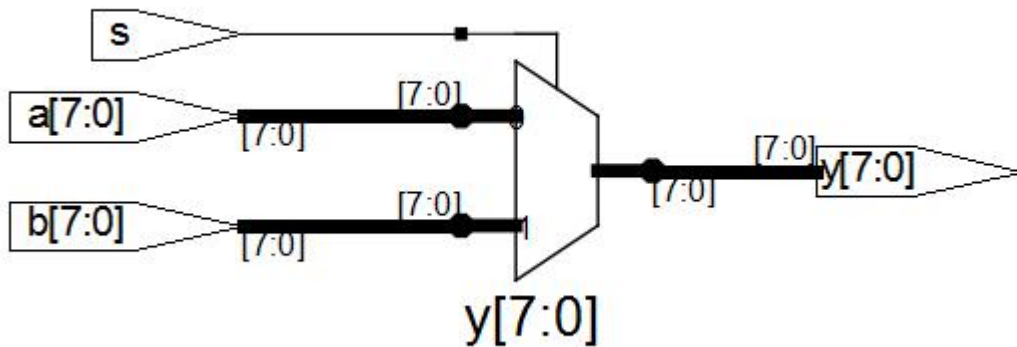
内部功能
定义




```

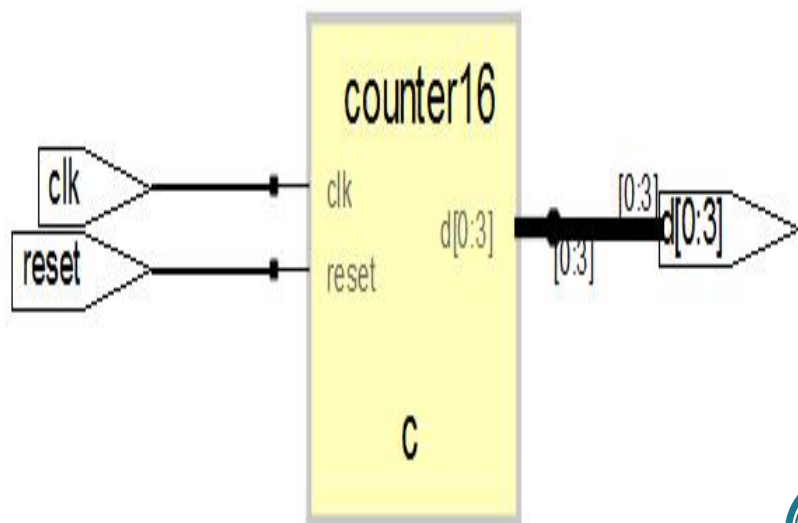
module mux2(a,b,s,y); //二路选择器
    input          s;      //I/O端口说明
    input[7:0]     a,b;
    output[7:0]    y;
    //功能定义
    assign y = (s==0) ? a : b;
endmodule

```



软件综合后的电路图图

二路选择器



```

module counter16( clk,reset,d );//模16计数器
    input          clk,reset;//输入输出端口声明
    output[0:3] d;
    reg[0:3] d; //声明变量d为寄存器类型

    always@(posedge clk) //功能定义
        if( !reset )
            d <= 0; //复位
        else
            d <= d + 1;
endmodule

```

软件综合后的电路图

模16计数器

Verilog HDL的特点

} 语法结构上的主要特点：

- 形式化地表示电路的行为和结构；
- 借用C语言的结构和语句；

与C语言
非常相似！

C与Verilog的意义不同

C语言

} 描述逻辑算法

} 语句顺序执行

Verilog

} 描述硬件

} 语句并行执行

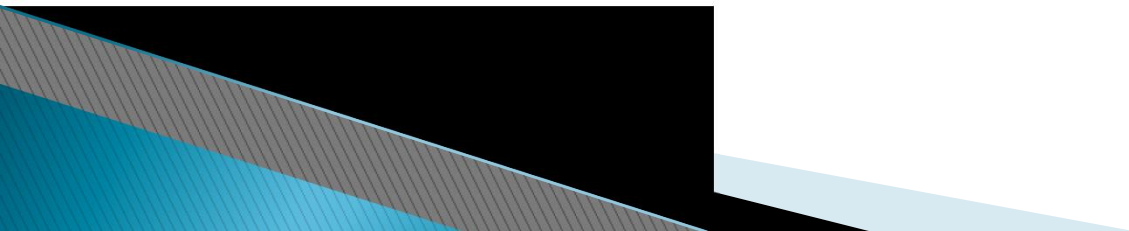
编写Verilog代码

} 词法

} (哪些单词可用)

} 语法

} (单词如何组织成有意义的语句)



Verilog的词法

- } 关键字 （用着用着就熟悉了）
- } 标识符 （与C语言类似）
- } 注释 （与C++相同） // /**/
- } 数字的写法

Verilog HDL关键字

and	edge	function	highz0	negedge
always	else	highz1		nor not
assign	end	if		notif0
begin	endcase	ifnone		notif1
buf	endfunction	initial		nmos
bufif0	endprimitive	inout		or
bufif1	endmodule	input		output
case	endspecify	integer		parameter
casex	endtable	join		pmos
casez	endtask	large		posedge
cmos	event	macromodule		primitive
deassign	for	medium		pulldown
default	force	module		pullup
disable		nand		pull0
				pull1

数字的写法

‘后的字母	意义
b	二进制
o	八进制
d	十进制
h	十六进制

书写格式：

<位数>’<进制>数字

负数在位数前加“-”

长数字之间可以用
下划线隔开

数字	位数	进制	值	存储
3'b101	3	二	5	101
'b101	-	二	5	0...000101
8'b1110_0001	8	二	225	11100001
-8'b1101	8	二	-13	11110011
8'd13	8	十	13	00001101
8'o15	8	八	13	00001101
8'h...	8	十六	13	00001101

特殊数字 x z

- x 表示不定值， z 表示高阻值；

- 例：

	与左栏等价
8'b0000_XXXX	8'h0x
8'b1100_ZZZZ	8'hcz

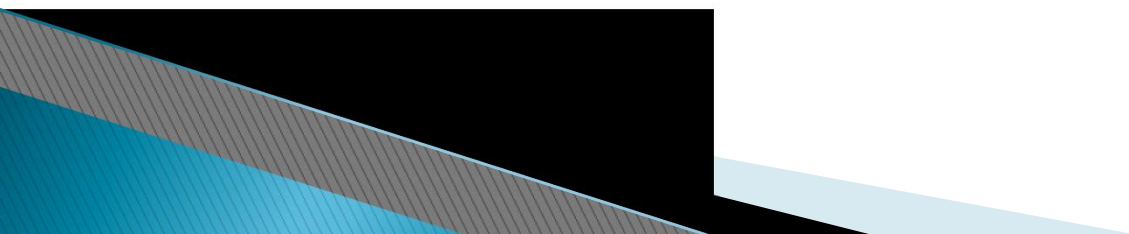
Verilog的语法

- } 常量、变量
- } 运算符及表达式
- } 语句

assign / = / if-else/case /语句块

- } 实例元件语句
- } 结构说明语句

always / initial



常量

- 在代码运行过程中，其值不能被改变的量，称为常量。
 - ◆ 数字（包括整数，x和z值，负数）
 - ◆ **parameter**常量（或称符号常量）

定义格式：

```
parameter 变量名1 = 数字,  
            变量名2 = 数字,  
            .....,  
            变量名n = 数字;
```

```
} 例: parameter UP      = 2' d0 ,  
      DOWN    = 2' d1 ,  
      LEFT    = 2' d2 ,  
      RIGHT   = 2' d3 ;
```

```
arrow = DOWN ;           //等价于  
arrow = 2' d1 ;
```

提高代码的
可读性

变量

} **wire**型变量

- 。总线

格式

wire 变量名1, 变量名2,, 变量名n;

wire[n-1:0] 变量名1, 变量名2,, 变量名m;

位宽为n

例: **wire** a; //声明**wire**型变量a,数据宽度1
 wire[7:0] b; //声明**wire**型变量b,数据宽度8

变量

reg 型变量具有记忆功能
wire型没有

} reg型变量

- 定义——在过程块中被赋值的信号，往往代表触发器，但不一定就是触发器（也可以是组合逻辑信号）！

格式

reg 数据名1, 数据名2,, 数据名n;

reg[n-1:0] 数据名1, 数据名2,, 数据名m;

每个变量
位宽为n

共声明了m
个reg型变量

```
例: reg      a; //声明reg型变量a,数据位宽1  
     reg[7:0] b; //声明reg型变量b,数据位宽8
```

变量

memory型变量——reg数组

定义——由若干个相同宽度的reg型向量构成的数组。

- } Verilog HDL通过reg型变量建立数组来对存储器建模。
- } memory型变量可描述RAM、ROM和reg堆。

`reg[n-1:0] 存储器名[m-1:0];`

每个存储单元位宽为n

共有m个存储单元

```
例: reg[7:0] RAM[63:0]; //声明存储器RAM
    RAM[10] = 8'd34; //访问第10个储存单元
    //看起来类似高级语言中的数组，但区别很大！
    //[10]的意义是地址
```


其他变量

} Verilog高级部分

- 其它数据类型：
large 型、medium 型、
scalared 型、small 型、
time 型、tri 型、tri0 型、
tri1 型、triand 型、trior
型、triereg 型、vectored
型、wand 型、wor 型等

运算符

- 一、算术运算符 (+, -, *, /, %)
- 二、逻辑运算符 (&& , || , !)
- 三、位运算符 (&, |, ~, ^, ^~)
- 四、关系运算符 (> , < , >= , <=)
- 五、赋值运算符 (<= , =)
- 六、移位运算符 (<< , >>)
- 七、条件运算符 (? :)
- 八、位拼接运算符 ({})

与C语言类似

拼接运算符（例）

```
} reg[0:7]  a_8 , b_8 ;  
} reg[0:15] a_16 , b_16 ;  
} reg[0:19] a_20 ;  
  
} always...begin  
}   a_16 = {a_8,b_8} ;  
}   a_20 = {a_8[7:4] , a_16 } ;  
}   {a_8,b_8} = 16'haacc ;  
} end
```

运算符的优先级

类 别	运 算 符	优先级
逻辑、位运算符	! ~	高  低
算术运算符	* / %	
	+ -	
移位运算符	<< >>	
关系运算符	< <= > >=	
等式运算符	= = ! = == = != =	
缩减、位运算符	& ~&	
	^ ^~	
	~	
逻辑运算符	&&	
条件运算符	? :	

➤ 为提高程序的可读性，建议使用**括号**来控制运算的优先级！

➤ **[例]** (a>b)&&(b>c)
(a==b)|| (x==y)
(!a)|| (a>b)

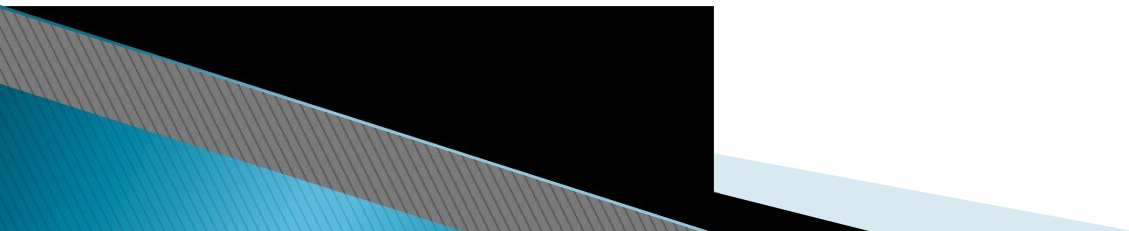
编写Verilog代码

} 词法

} (哪些单词可用)

} 语法

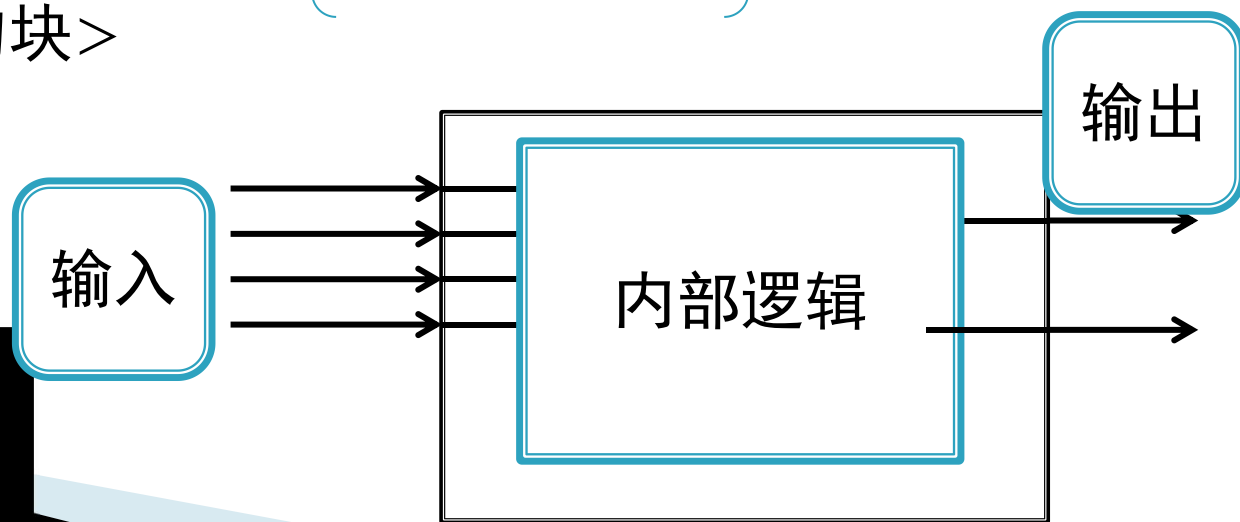
} (单词如何组织成有意义的语句)



Verilog模块基本结构

```
} module <模块名>(口1, 口2, .....);  
}   <I/O端口说明>  
}  
}   .....  
}   <内部信号说明>  (变量声明)  
}  
}   .....  
}   <assign语句>  
}   <实例元件语句>  
}   <结构说明语句块>  
} endmodule
```

内部功能
定义



} assign语句：无论右边表达式操作数何时发生变化，右边表达式都会重新计算，并且给左边变量赋值。

格式

assign <变量> = <表达式> ;

模块中的I/O端口类型
缺省为wire型。

```
module myand(a,b,y);  
    input a,b;  
    output y;  
    assign y = a & b;  
endmodule
```

一般用于赋值wire
型变量

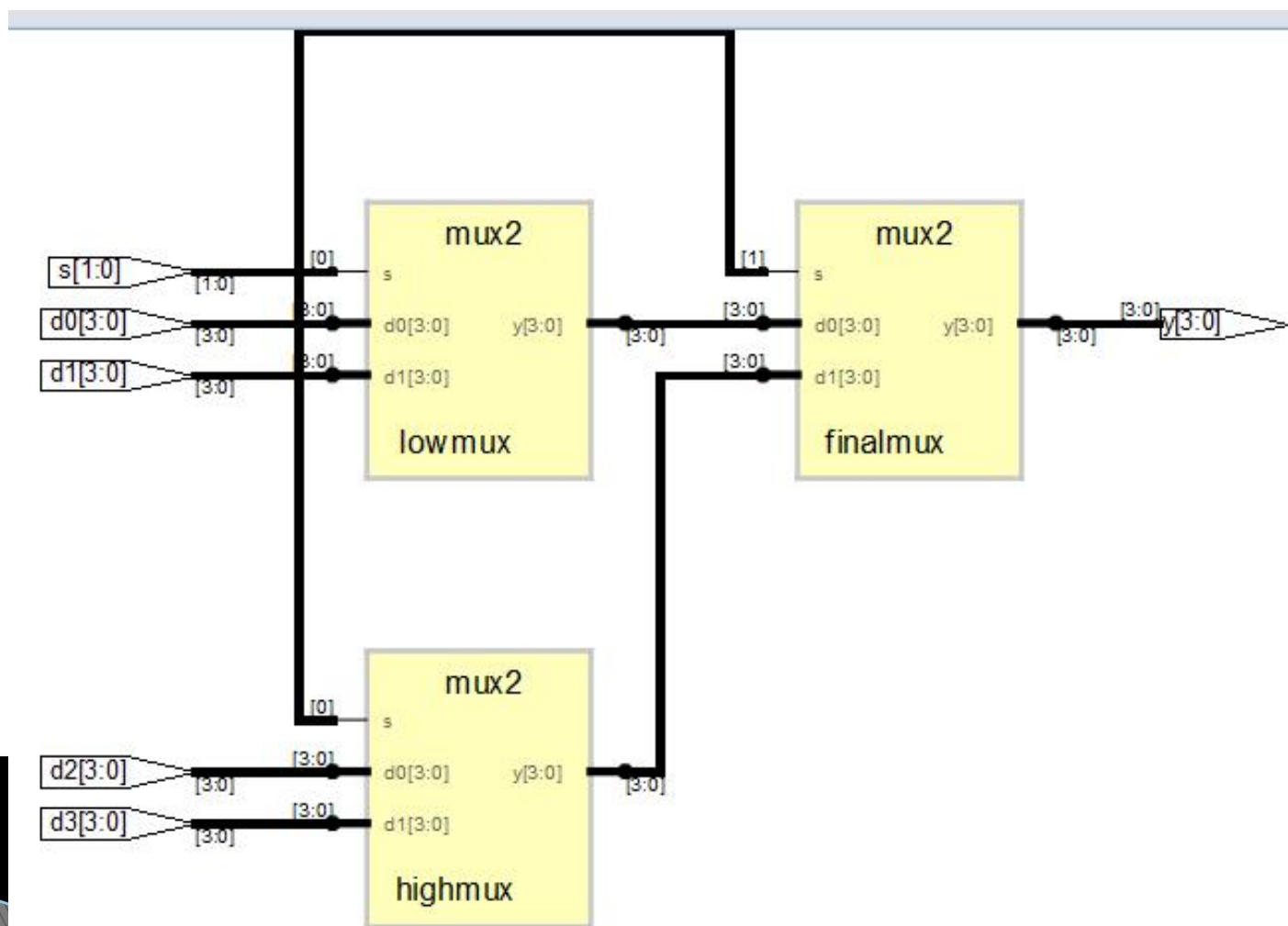
assign语句一直执行，
因而也被称为连续赋值
语句

实例元件或子模块语句

格式

模块名 实例名 (口1, 口2, ……)

} 例:
} 四路选择器



实例元件或子模块语句

格式

模块名 实例名 (口1, 口2, ……)

} 例：四路选择器

```
module mux4 (d0,d1,d2,d3,s,y);  
    input[3:0]  d0,d1,d2,d3;  
    input[1:0]   s;  
    output[3:0] y;  
  
    wire[3:0] low,high;  
  
    mux2 lowmux(d0,d1,s[0],low);  
    mux2 highmux(d2,d3,s[0],high);  
    mux2 finalmux(low,high,s[1],y);  
endmodule
```

参数信号必须与
底层模块一一对
应

结构说明语句

} always 块 （有触发条件）

```
always @ ( 敏感信号列表)  
    <语句>
```

} initial 块 （没触发条件，只在仿真零时刻执行一次）

```
initial  
    <语句>
```

always 块

```
always @(a or b) // 每当a或b的值变化时执行一遍块内语句  
begin  
    <语句> .....  
end
```

电平触发

```
always @(posedge clk) // 每当时钟上升沿到来时执行一遍块内语句  
begin  
    <语句> .....  
end
```

也可以是信号沿
触发,只能一位

当敏感信号是沿触发
时, 那么该always块
描述的是时序电路

initial 块

◆与always块的区别：无触发条件只在仿真零时刻执行一次

◆一般用于测试激励文件

} 例：

```
module test;
    wire a , b ;
    wire y ;
    initial begin
        a = 0; b = 1;
        #10 a = ~a; b = ~b;
    end
    myand myand(a,b,y);
endmodule
```

```
module myand(a,b,y);
    input a,b;
    output y ;
    assign y = a & b;
endmodule
```

语句

能在**always**
和**initial**块中
使用

} 赋值语句

 <表达式> =|<= <表达式>;

} 条件语句

if<表达式><语句1>

else<语句2>

if和**case**只能
在**always**块
中使用

} 多分支选择语句

case(表达式)

 分支1; 分支2; **default**:

endcase

} 块语句

begin <语句> **end**

语句（例）赋值语句

例：

```
a_8 = {b_8[7:4] , c_8[3:0]} ;  
a_8 = b_16[ 8: 1] ;  
  
b_16[ 15:8 ] = a_8 ;
```

不建议将一个
reg变量拆开分
别赋值，会出
现问题

注意！ 与assign的区别：

assign 连续赋值：等号左边恒等右边

always块内赋值：在**always**触发时右边复制给左边

赋值只能写在在**always / initial**块中，左边是**reg**变量；

assign 语句只能写在**always**外，左边是**wire**变量，与**always**块并发执行

阻塞赋值与非阻塞赋值

阻塞赋值 =

非阻塞赋值 <=

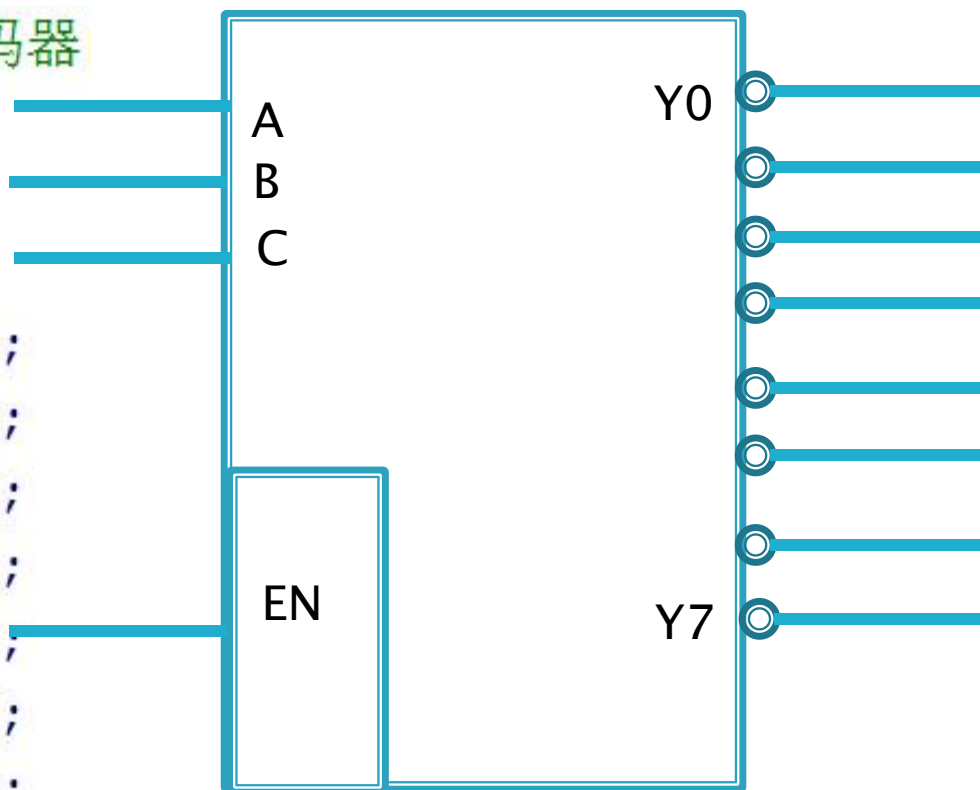
} 描述组合逻辑电路

} 描述时序电路

(例) 3-8译码器

```
module dec3_8(a,y);//3-8译码器
    input [2:0] a;
    output[7:0] y;

    assign y[0] = (a==3'b000);
    assign y[1] = (a==3'b001);
    assign y[2] = (a==3'b010);
    assign y[3] = (a==3'b011);
    assign y[4] = (a==3'b100);
    assign y[5] = (a==3'b101);
    assign y[6] = (a==3'b110);
    assign y[7] = (a==3'b111);
endmodule
```



(例) 3-8译码器

```
module dec3_8(a,y); //3-8译码器
    input [2:0] a;
    output [7:0] y;

    reg [7:0] yy;
    assign y = yy;
    always @( a )begin
        case( a )
            3'b000: yy = 8'b0000_0001;
            3'b001: yy = 8'b0000_0010;
            3'b010: yy = 8'b0000_0100;
            3'b011: yy = 8'b0000_1000;
            3'b100: yy = 8'b0001_0000;
            3'b101: yy = 8'b0010_0000;
            3'b110: yy = 8'b0100_0000;
            3'b111: yy = 8'b1000_0000;
        endcase
    end
endmodule
```

yy在always中被
赋值，必须是
reg型

(例) 3-8译码器

```
module dec3_8 (a, y); // 3-8译码器
    input [2:0] a;
    output [7:0] y;

    assign y = 1 << a;
endmodule
```



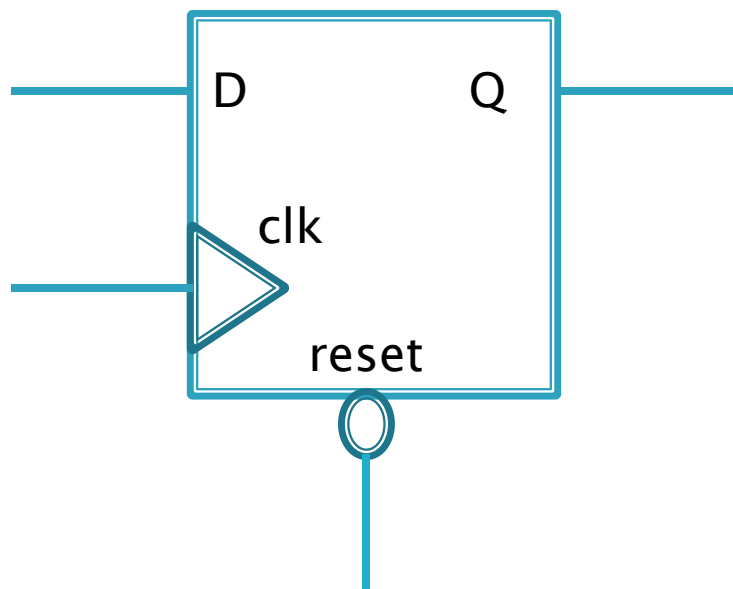
行为描述
抽象级别更高

带使能译码器

```
module dec3_8 (a,en,y) ;//3-8译码器带使能
    input      en;
    input [2:0] a;
    output [7:0] y;

    assign y = en ? (8'b1 << a) : 8'b0;
endmodule
```

(例) 可复位寄存器



语句（例）可复位寄存器

```
module flopr(clk,reset,d,q); //可复位寄存器
    input clk,reset;
    input[3:0] d;
    output[3:0] q;
    reg[3:0] q; //赋值语句左边必须是reg型变量
```

```
    always @(posedge clk or nedge reset)
        if(!reset) q <= 4'b0; //reset为零, 复位
        else q <= d; //否则把d的值复制给q
```

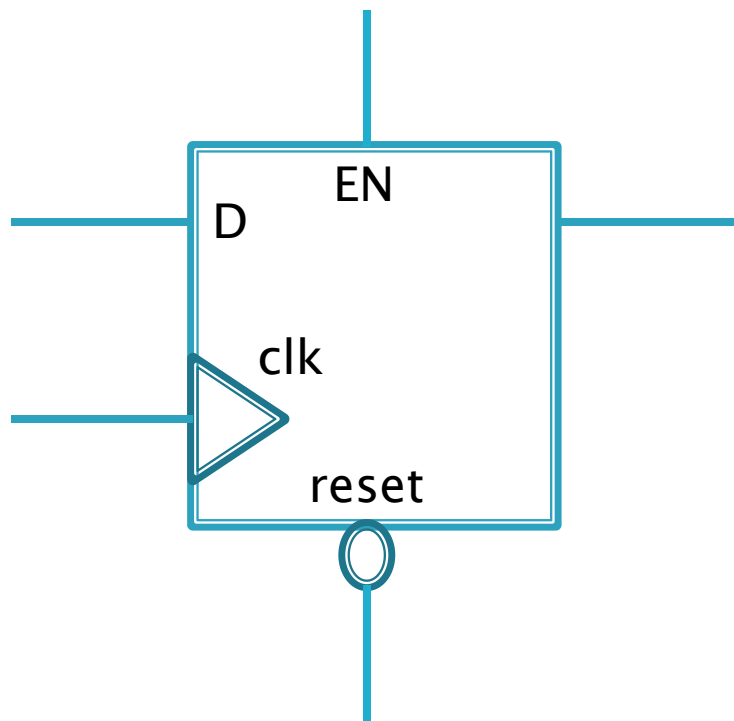
```
endmodule
```

always块中不能使用
assign赋值
只能使用 = / <= 赋值
语句

时序电路用

<=

(例) 带使能触发器



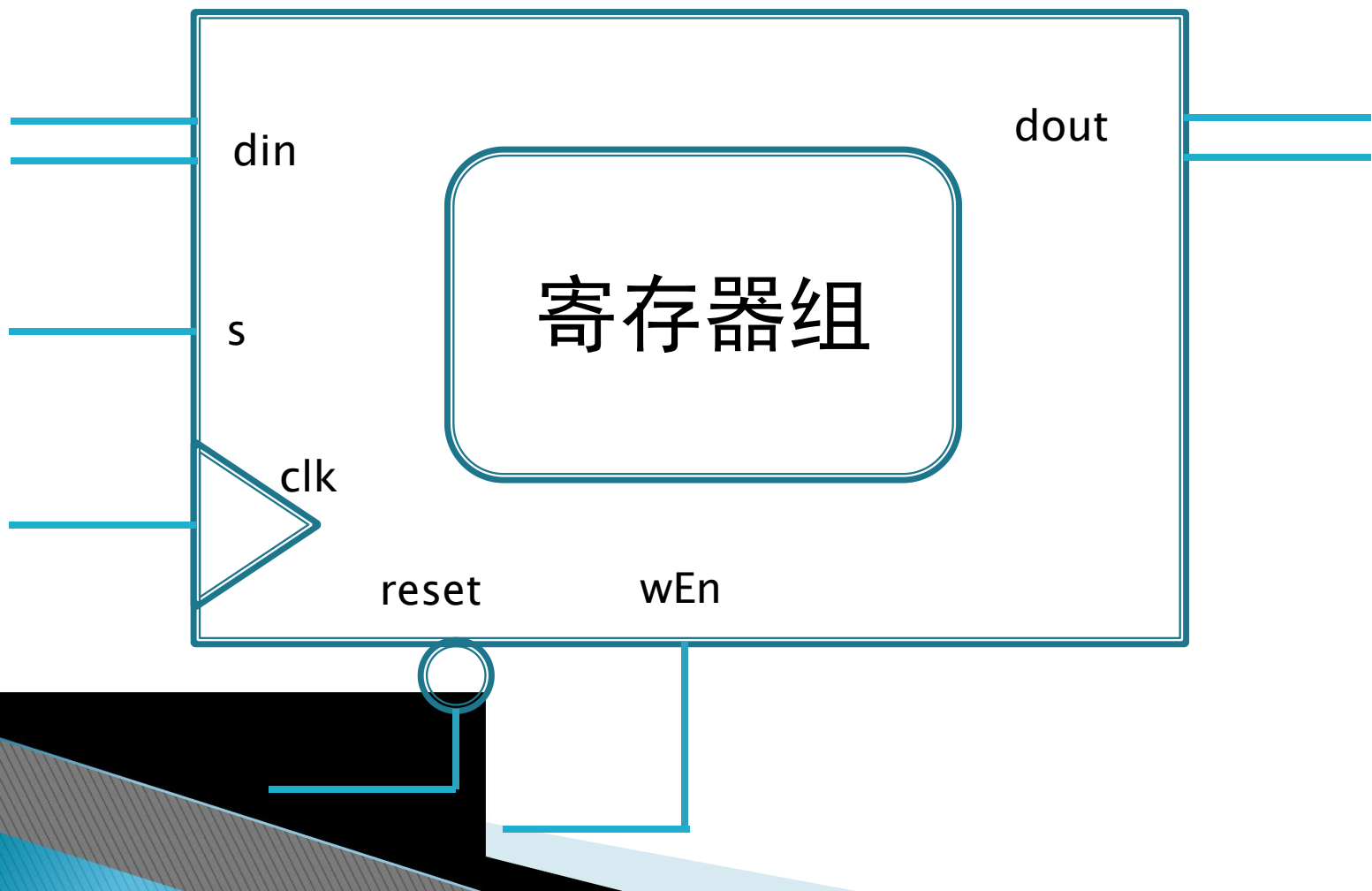
(例) 带使能触发器

if 语句的嵌套使用

```
module flopr(clk,reset,En,d,q); //带使能的4位寄存器
    input clk,reset,En;
    input[3:0] d;
    output[3:0] q;
    reg[3:0] q; //赋值语句左边必须是reg型变量

    always @(posedge clk or nesedge reset)
        if(!reset) q <= 4'b0; //reset为零, 复位
        else if(En) q <= d; //否则当En为高电平时
                           //把d的值复制给q
endmodule
```

(例) 寄存器组



(例) 寄存器组

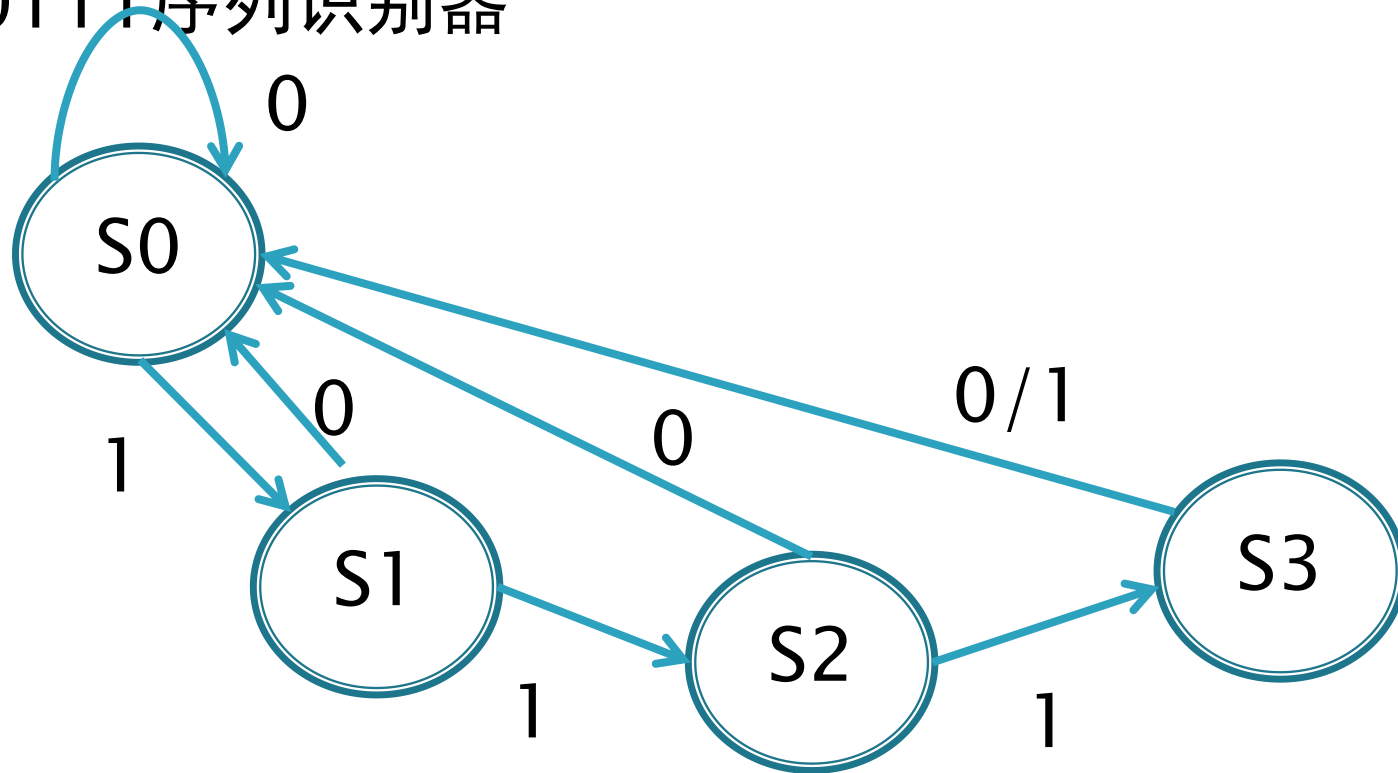
```
module regFile( clk,reset,s,wEn,din,dout);  
    input      clk,reset,wEn;  
    input[1:0]  s;  
    input[7:0]  din;  
    output[7:0] dout;  
  
    reg[7:0]    R[0:3];  
  
    assign dout = R[s];  
  
    always@( posedge clk or negedge reset )  
        begin  
            if( !reset ) begin //复位  
                R[0] <= 0;      //这是时序电路  
                R[1] <= 1;      //使用"<="   
                R[2] <= 2;  
                R[3] <= 3;  
            end  
            else if( wEn ) R[s] <= din;  
        end  
endmodule
```

类似数组的访问
assign必须写在
always外

这里dout不是时
序的

(例) 状态机

} 0111序列识别器

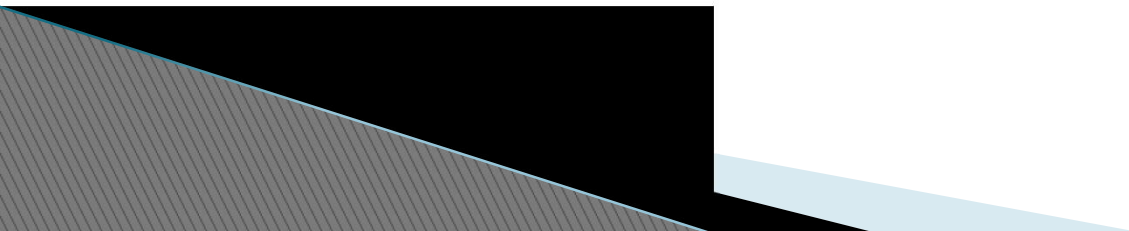


(例) 状态机

```
module fsm(clk,in,out); //0111序列识别器
    input  clk,in;
    output out;
    reg[1:0] state;      //当前状态
    parameter s0 = 2'b00,
               s1 = 2'b01,
               s2 = 2'b10,
               s3 = 2'b11;

    assign out = (state==s3); //当状态是s3,输出1, 否则输出0
    initial state = s0;      //初始状态s0
    always@(posedge clk)begin //时钟上升沿触发
        if(in==0) state <= s0;
        else      state <= state + 1;
    end
endmodule
```

- } 这里讲的只是Verilog的一些基本功能
- } Verilog的高级功能需要自己继续学习、体会



复习(概念)

- } 词法
 - } 关键字，标识符，数字，运算符
- } 语法
 - } **wire**, **reg**, reg数组(寄存器组、存储器)
 - } **initial**, **assign**, **always**, 实例元件语句
 - } 赋值、**if-else**、**case**
- } 数据流描述、行为描述

复习(模块)

```
} module name(ports);  
    I/O ports;  
    inner signals;  
    initial...;  
    assign...;  
    always@()  
        assignments;  
    if-else;  
    case;  
} endmodule
```



正确地描述你的硬件

组合逻辑电路

```
} assign  
} always@ (变量)  
} begin  
} if-else语句  
} case语句  
} 阻塞赋值语句 =  
} end
```

时序电路

```
} always@( edge 时钟)  
} begin  
} if-else语句  
} case语句  
} 非阻塞赋值语句 <=  
} end
```

描述基本器件

组合逻辑电路

- } 多路选择器
- } 编码器
- } 译码器
- } 运算器

时序电路

- } 寄存器
- } 寄存器组
- } 计数器
- } 状态机

复杂电路

- } 实例基本器件
- } 逻辑语句

MIPS单周期CPU数据通路

复杂的数字系统都是由简单的器件组成的

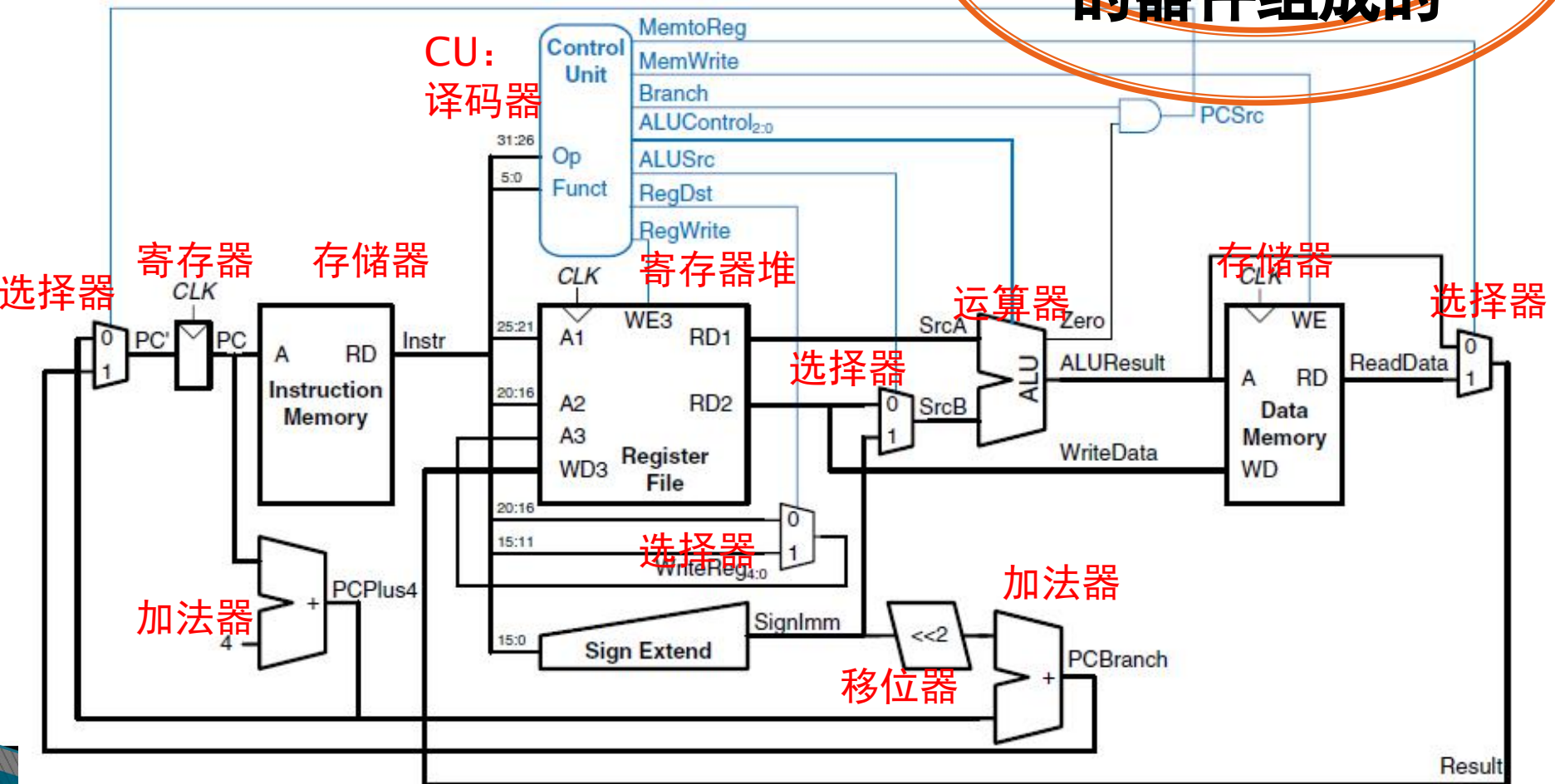


Figure 7.11 Complete single-cycle MIPS processor

带使能译码器

```
module dec3_8 (a,en,y) ;//3-8译码器带使能
    input      en;
    input [2:0] a;
    output [7:0] y;

    assign y = en ? (8'b1 << a) : 8'b0;
endmodule
```

带参数的二路选择器

//带参数的二路选择器

```
module mux2 #(parameter WIDTH = 8)  
    (a,b,s,y);  
  
    input                s;  
    input [WIDTH-1:0] a,b;  
    output [WIDTH-1:0] y;  
  
    assign y = (s==0) ? a : b;  
endmodule
```

//带参数实例

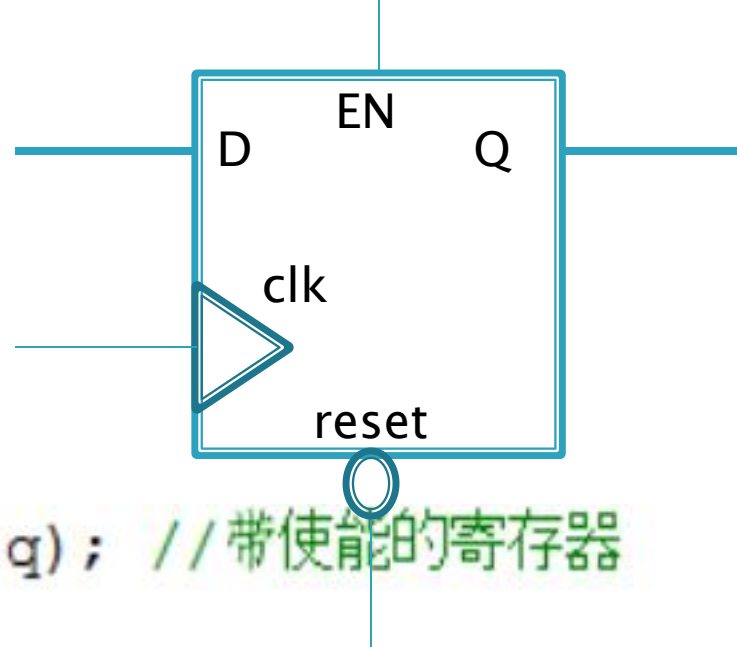
```
mux2 #32 m1(a,b,s,y); //32位  
mux2 m2(a,b,s,y); //默认8位
```


时序电路

- } 只在时钟到来时才改变
- } 有记忆功能

- } 寄存器、寄存器堆
- } 计数器、有限状态机

寄存器



```
module flopr(clk,reset,En,d,q); //带使能的寄存器
    input clk,reset,En;
    input[3:0] d;
    output[3:0] q;
    reg[3:0] q; //赋值语句左边必须是reg型变量

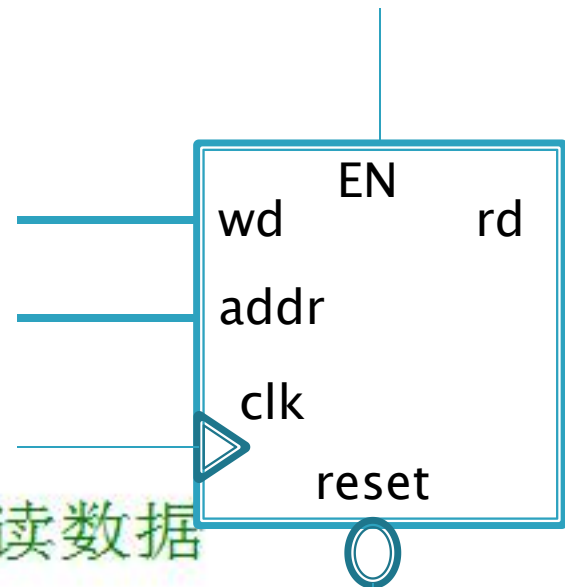
    always @(posedge clk or nesedge reset)
        if(!reset) q <= 4'b0; //reset为零, 复位
        else if(En) q <= d; //否则当En为高电平时
                           //把d的值复制给q

endmodule
```

寄存器堆 (多个寄存器堆在一起,一时刻只使用一个)

```
module regfile (clk, wEn, addr, wd, rd); //寄存器组
    input        clk, wEn;
    input[1:0]    addr;
    input[7:0]    wd;
    output[7:0]   rd;
    reg[7:0]      regfile[0:4];

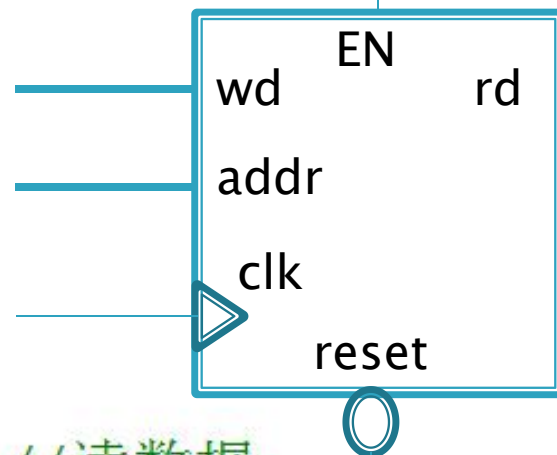
    assign rd = regfile[addr]; //读数据
    always @ (posedge clk )
        if( wEn ) regfile[addr] <= wd; //写数据
endmodule
```



寄存器堆 (多个寄存器堆在一起, 一时刻只使用一个)

```
module RegFile( clk,reset,wEn,addr,wd,rd); //寄存器堆
    input      clk,reset,wEn;
    input[3:0]  addr;
    input[7:0]  wd;
    output[7:0] rd;

    reg[7:0] regfile[0:15];
    integer i;
    assign rd = regfile[addr]; //读数据
    always@(posedge clk or negedge reset)begin
        if(!reset)
            for(i = 0;i < 16;i = i + 1;) //for语句复位
                regfile[i] <= 8'b0; //方便的写法
        else
            if(wEn) regfile[addr] <= wd; //写数据
    end
endmodule
```



计数器

(可以计数的寄存器)

```
module counter16( clk,reset,d );//模16计数器
    input          clk,reset;
    output[0:3]    d;
    reg[0:3]       d;

    always@ (posedge clk)
        if( !reset )
            d <= 0;
        else
            d <= d + 1;
endmodule
```